Sampler

115.04.15

大綱

- LLM 介紹
  - LLM 是什麼
  - LLM 怎麼運作
  - LLM 的輸入&輸出
  - LAB-1
- LLM 的生成機率分布
- LLM 的 token 挑選秘辛
  - temperature
  - LAB-2
  - topK, topP, minP
  - LAB-3
  - random, greedy
  - LAB-4
  - penalty
  - LAB-5
- LLM 的挑戰
- LAB-Summary

LLM 介紹

LLM 是什麼

- Large Language Model 大型語言模型
- 利用大量資料預訓練的深度學習模型，可以理解並生成近似人類語氣的語言
- 應用
  - 內容生成: 行銷文件、郵件
  - 自然語言理解與處理: 文章摘要、情感分析、語言翻譯
  - 問答系統與對話: 客服機器人 (ChatBot) e.g. Chat-GPT
  - 程式碼撰寫: 程式編寫建議與優化 e.g. Github Copilot

LLM 的運作(1/2)

- 我們的想像: 我們問一句，LLM 回答一句

INPUT

今天天氣如何→

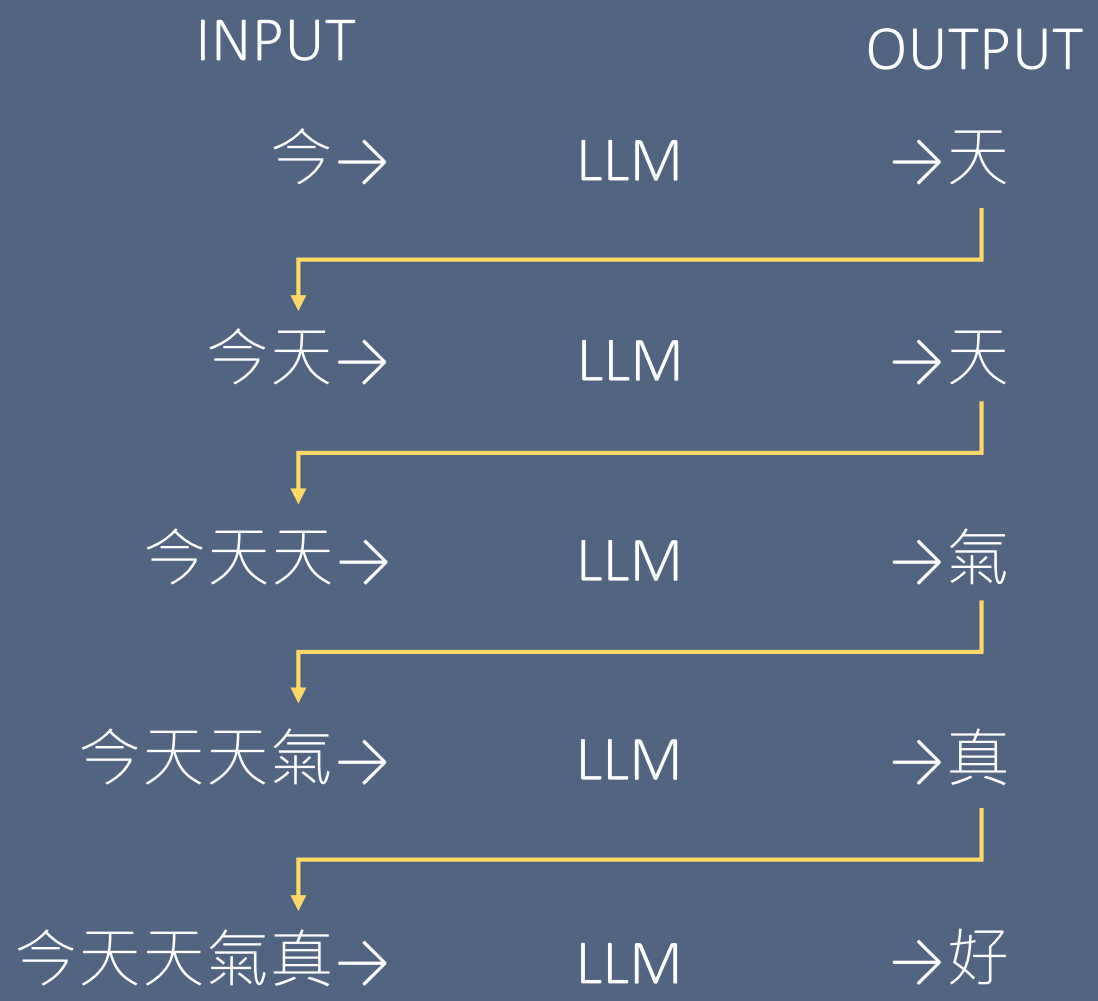
LLM

OUTPUT

→今天天氣真好

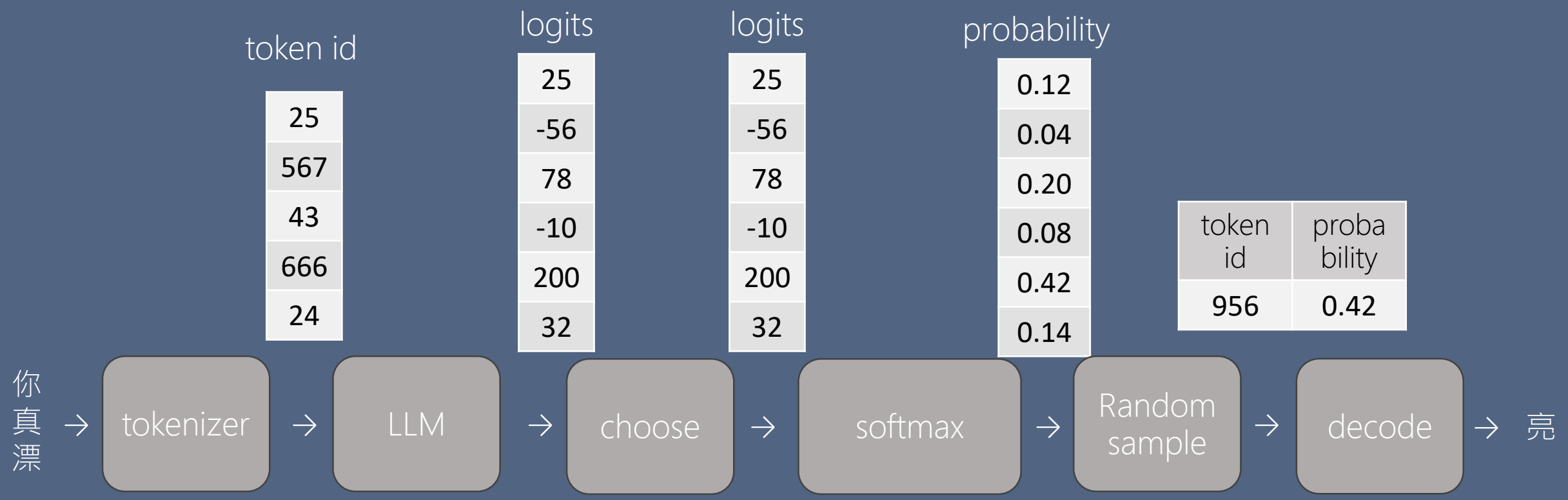
LLM 的運作(2/2)

- 實際上: 迭代生成
- 好處
  - 即時生成控制
  - 簡化問題

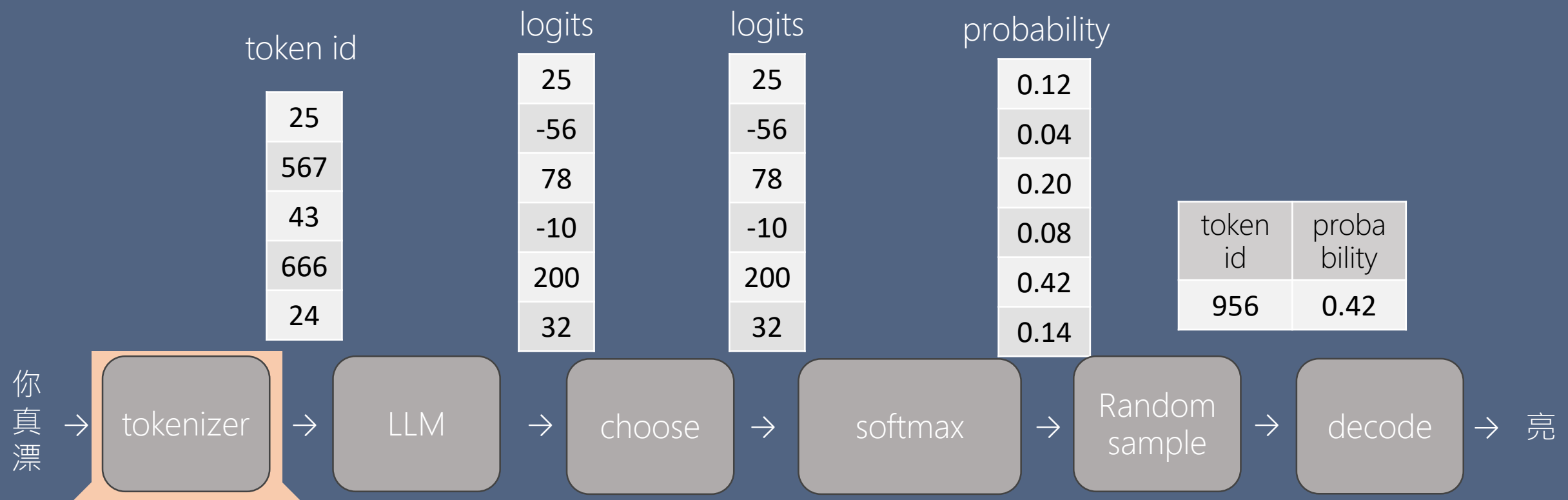


GET: 今天天氣真好

LLM 的運作流程


token id
25
567
43
666
24
logits
25
-56
78
-10
200
32
logits
25
-56
78
-10
200
32
probability
0.12
0.04
0.20
0.08
0.42
0.14
token id
probability
956
0.42
你
真
漂
tokenizer
LLM
choose
softmax
Random sample
decode
亮

LLM 的輸入 (1/6)



- Tokenize
- 轉成 token id
- 加入 special token

LLM 的輸入 (2/6)

- 模型只能處理數字，所以需要將文字處理成模型看得懂的樣子

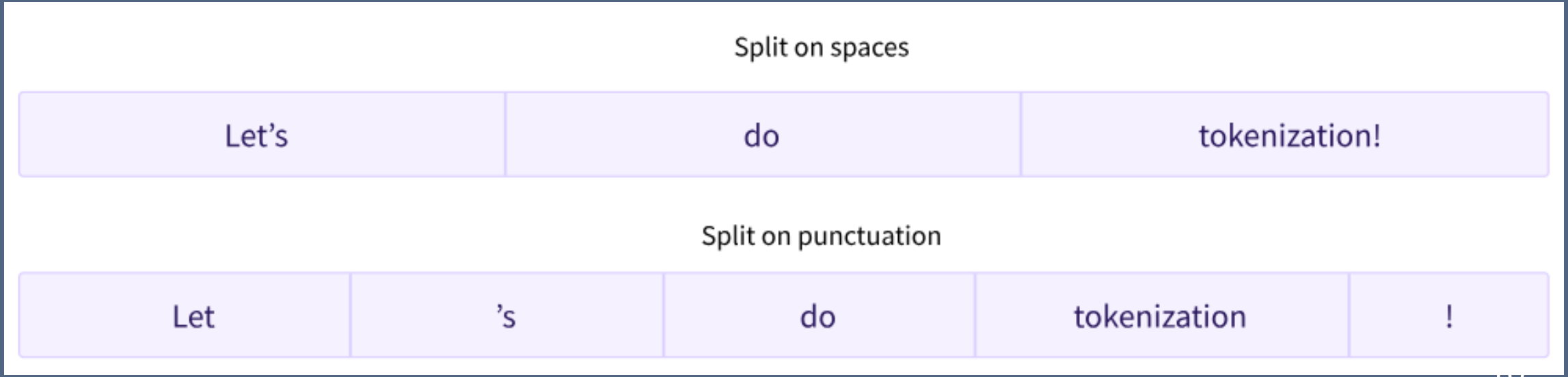
- 將文字以最有意義的方式轉換成模型看得懂的樣式 → Tokenizer 的目標
  1. 切分成 token (tokenize)
     - 不同模型可能有不同的 tokenize 方式
       - Word-based
       - Character-based
       - Subword-based 最常見!
  2. 再轉換成數字 (token id)
     - 透過 mapping 找到 token 對應的代碼
  3. 加入用來給模型辨識的特殊字符 (special token)
     - 提示模型文字的特殊內容 e.g. 起始&結束

今天天氣真好
↓
Tokenizer
↓
[235, 461, 461, 20, 158, 9]

LLM 的輸入 (3/6)

- Word-based
- 拆分為單字

優點	缺點
• 簡單	• 會有大量標記，因為不同單字都有不同轉換結果 e.g. dog 和 dogs 會被視為不相關的



https://huggingface.co/learn/llm-course/zh-TW/chapter2/4

LLM 的輸入 (4/6)

- Character-based
- 拆分為字符

優點	缺點
• 詞彙量極小	• 字符的意義低 • 一個字可能被拆成多個 token

https://huggingface.co/learn/llm-course/zh-TW/chapter2/4

LLM 的輸入 (5/6)

- Subword-based
- 根據意義拆分

- 節省詞彙空間，同時也保留拆分意義

最常見!

Let's </w>	do</w>	token	ization</w>	!</w>

https://huggingface.co/learn/llm-course/zh-TW/chapter2/4

LLM 的輸入 (6/6)

- Special token

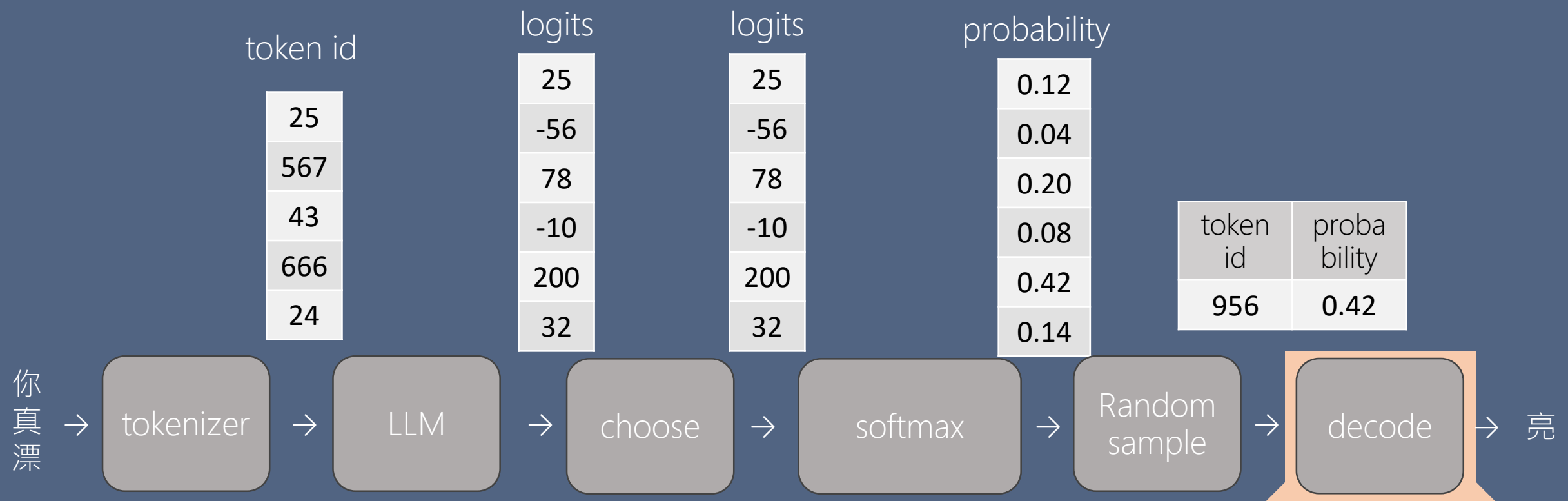
- 模型是利用這些轉換過的 id 進行訓練
- 需要一些特殊提示 → 讓推理時可以維持一樣的效果

- 根據模型不同而不同
- e.g. BERT 模型
  - [CLS] 放句首可提示模型進行分類
  - [SEP] 用來區隔句子

  - → [CLS] 今天天氣真好 [SEP]

https://huggingface.co/learn/llm-course/zh-TW/chapter2/6

LLM 的輸出



- 透過 tokenizer 將 token id 轉成 token

LAB-1

- 目標: 了解模型推論過程及必備操作

- PART1: 生成
  - 直接使用 library 進行完整生成 > 會一次得到完整的句子
  - 迭代生成 > 觀察一個字一個字生成的情境

- PART2: tokenizer 使用
  - 使用 library 一次 tokenize 完畢
  - 依照各功能分開 call library，觀察效果

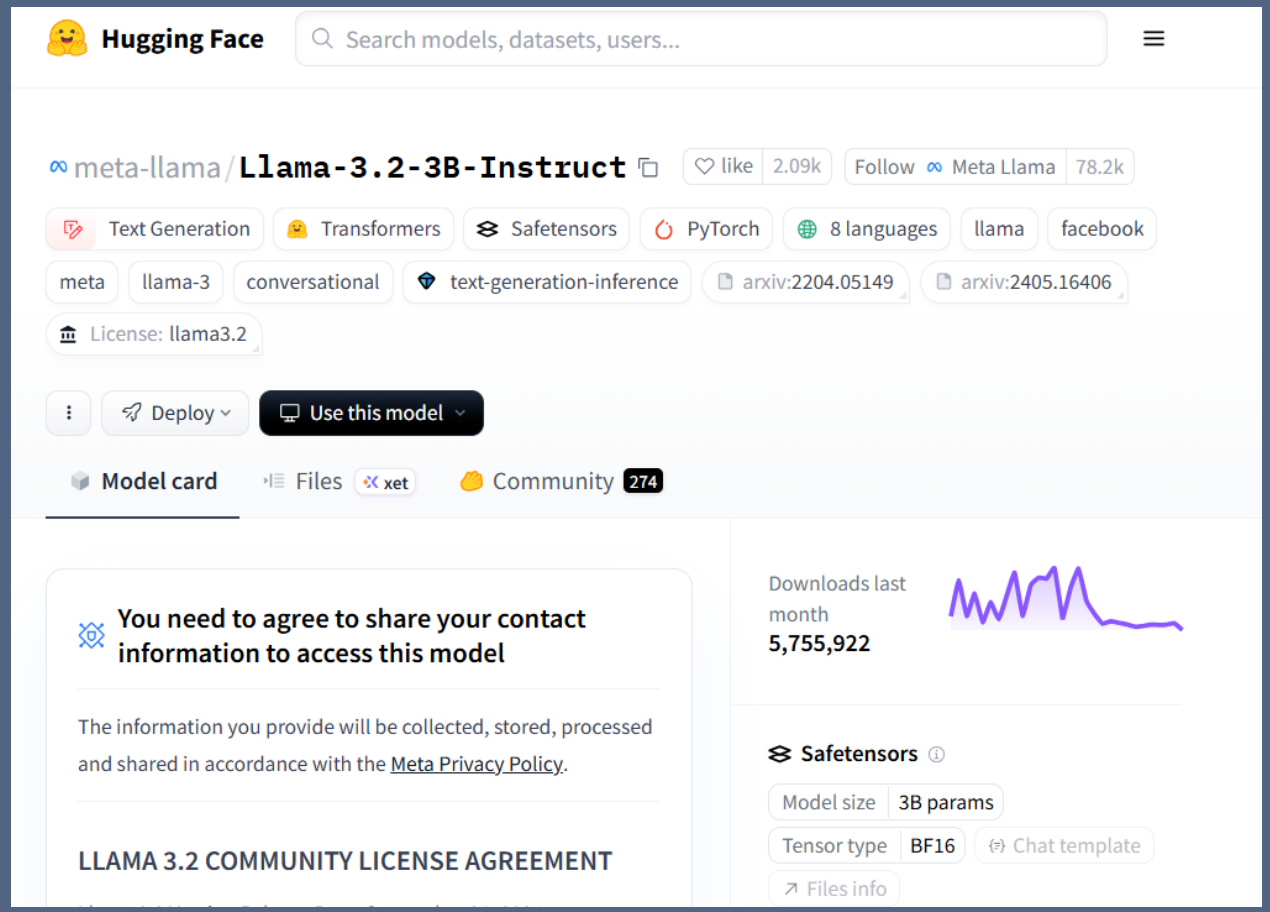
LAB-1

- Transformers 是開源的深度學習函式庫
- Framework
  - 基於 pytorch
- 提供
  - 模型套件
  - 操作的統一 API
  - 模型分享社群
    - 解決了 pretrained model 需大量資料和資源進行訓練的問題
    - 使用者可以用較少量資源進行 finetune

https://huggingface.co/docs/transformers/installation

LAB-1

- 使用 Llama-3.2-3B-Instruct
- 發表公司為 Meta
- 3B (30億) 個參數
- 好處
  - 可以透過 huggingface 進行使用
  - 速度快，資源需求低 (模型輕量)



https://huggingface.co/meta-llama/Llama-3.2-3B-Instruct

LAB-1

環境安裝

- 開啟 container

  - sudo docker run --gpus all -it --ipc=host --ulimit memlock=-1 --ulimit stack=67108864 -v {model path on host}:/model -v {work path on host}:{work path on container} pytorch/pytorch:2.9.0-cuda12.8-cudnn9-runtime

- (optional) 如果想用 terminal 方式編輯內容，可以安裝編輯套件

  - apt-get update
  - apt-get install nano

- 安裝所需套件

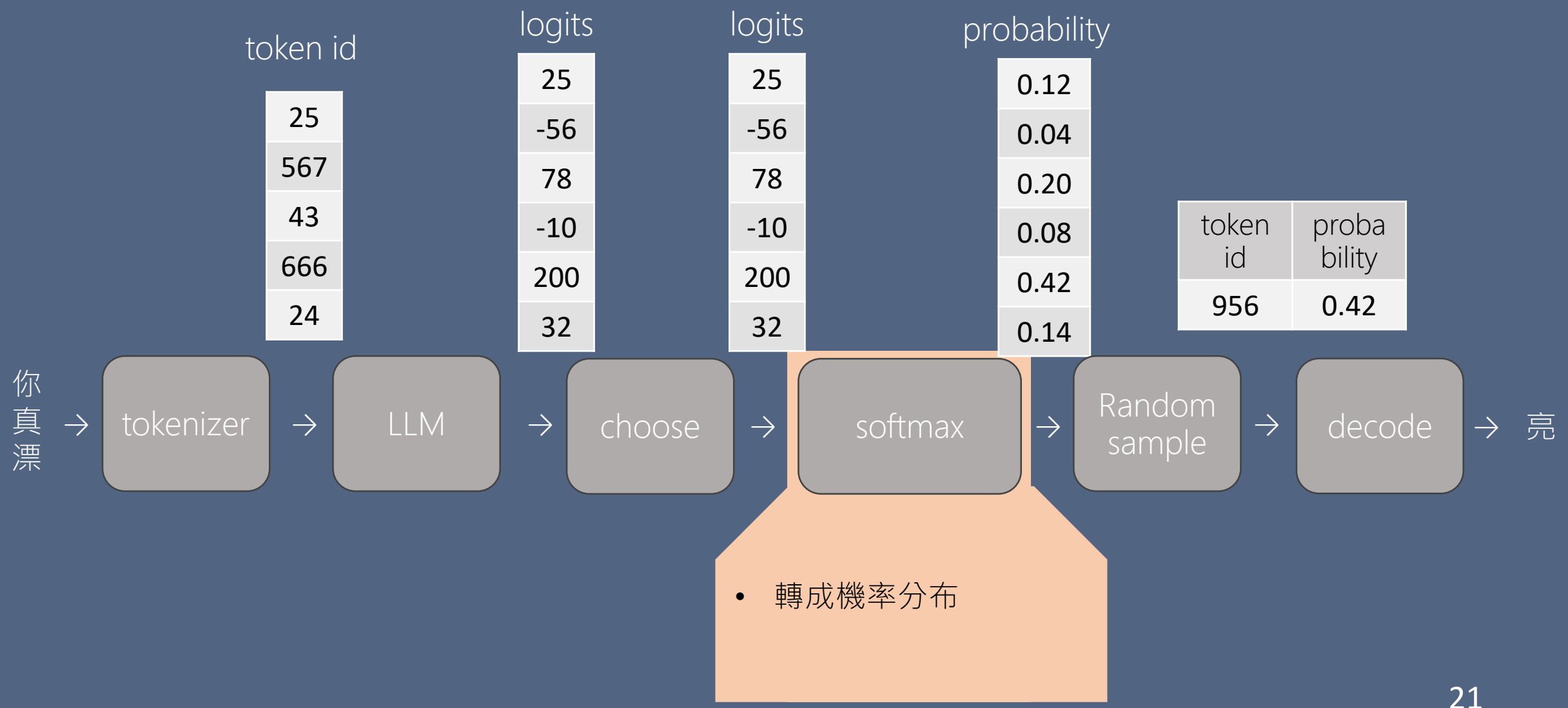
  - pip install transformers==4.57.3
  - pip install accelerate==1.12.0

LAB-1

- Code 架構介紹

LLM 的生成機率分布

LLM 的流程位置


token id
25
567
43
666
24
logits
25
-56
78
-10
200
32
logits
25
-56
78
-10
200
32
probability
0.12
0.04
0.20
0.08
0.42
0.14
token id
probability
956
0.42
你
真
漂
tokenizer
LLM
choose
softmax
Random sample
decode
亮
• 轉成機率分布

模型的輸出

- 模型會決定誰是下個產出的 token
- 實際上是在幫字典裡的 token 打分數，這些分數就叫 logits
- 分數可能有正有負

今天天氣真→ LLM →

token	logits
好	2.0
晴	1.0
壞	0.5
庫	0.0
盤	-0.5
腦	-1.0
狗	-2.0

為什麼不能直接用 logits

- 如果直接選分數最高的會發生什麼事?
- → 每次都只會選到同一個 token

- 今天天氣真好
- 今天天氣真好
- 今天天氣真好
- …

今天天氣真→ LLM →

token	logits	
好	2.0	100%
晴	1.0	0%
壞	0.5	0%
庫	0.0	0%
盤	-0.5	0%
腦	-1.0	0%
狗	-2.0	0%

沒有多樣性!

Softmax

- 將分數化為機率分布，使機率低的也有低機會被挑到
- 特性
  - 總和為 1
  - 高 logits 會轉為高機率、低 logits 轉為低機率

已經可以被拿來挑選 token 了!

token	logits
好	2.0
晴	1.0
壞	0.5
庫	0.0
盤	-0.5
腦	-1.0
狗	-2.0

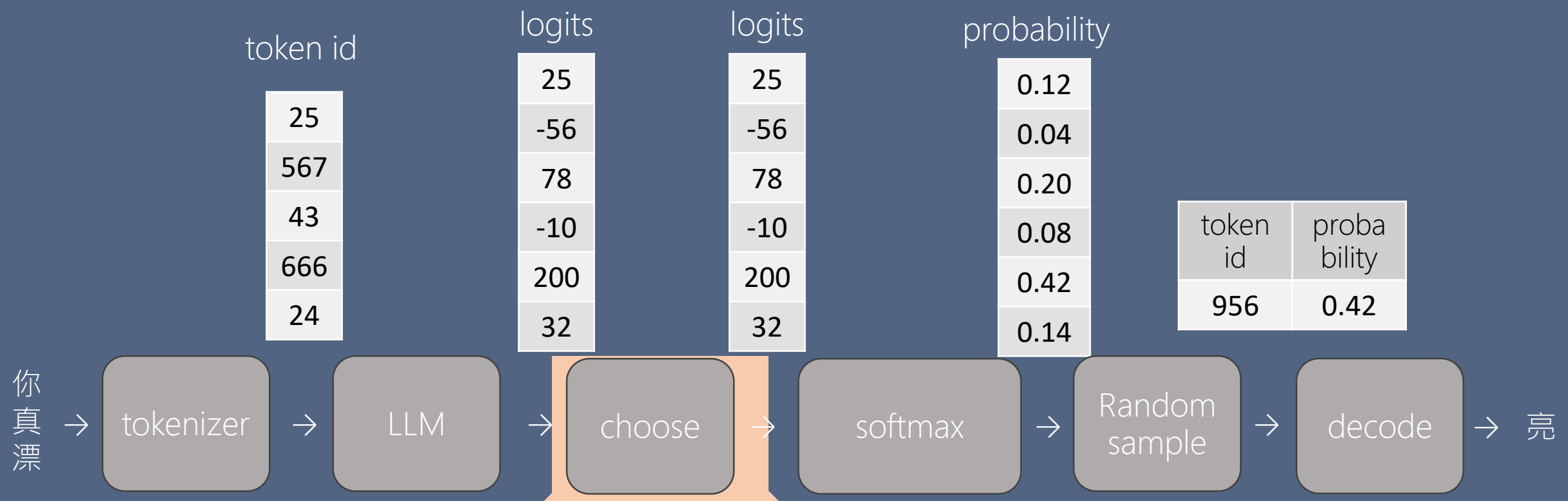
SOFTMAX

$$\rightarrow \quad Softmax(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}} \quad \rightarrow$$

probability
0.533
0.196
0.119
0.072
0.044
0.027
0.010

LLM 的 token 挑選秘辛

LLM 的流程位置



- 用 temperature 微調
- Penalty
- topK, topP, minP (softmax needed)
- Random, greedy

LLM 的 token 挑選秘辛

temperature

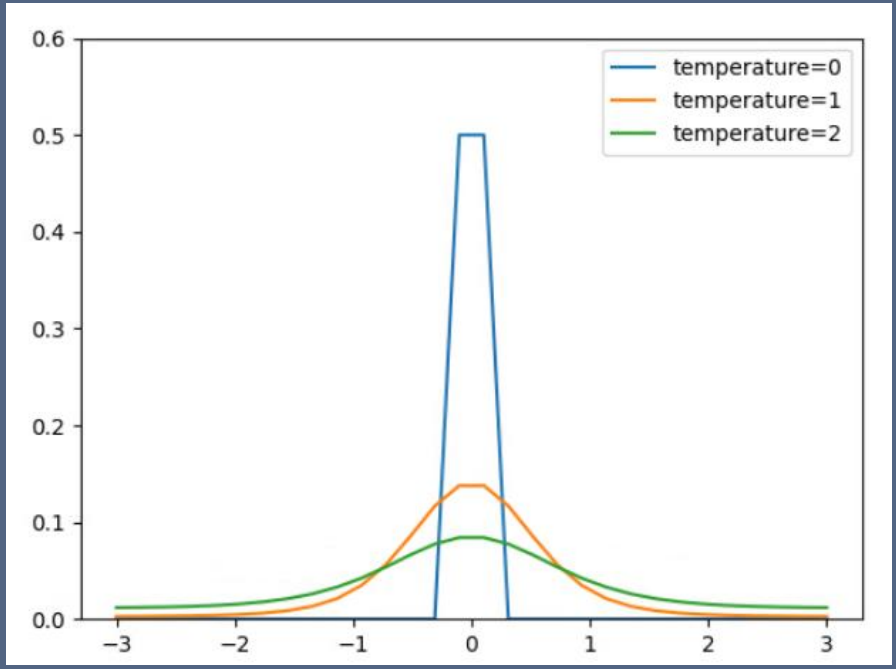
Temperature

- Softmax: 將 logits 轉換成機率分布
- Temperature: 調整 softmax 轉換的機率分布

$$P(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}} \quad \rightarrow \quad \text{加入 Temperature T} \quad \rightarrow \quad \mathrm{P}(x_i) = \frac{e^{\frac{x_i}{\mathrm{T}}}}{\sum_j e^{\frac{x_j}{\mathrm{T}}}}$$

較高的 T
→分布平坦→多樣性高

較低的 T
→分布尖銳→多樣性高



LAB-2

- 目標: 了解 softmax 並觀察不同 temperature 設定

- 請實作 softmax，並且加入 temperature 設定
- 嘗試用不同 temperature 對下方 logits 進行轉換
  - Temperature= 0, 1, 2

token	logits
A	3.0
B	1.0
C	0.0
D	-1.0
E	-2.0

LAB-2 分析

- 轉換右方的 logits
  - T=2 機率分布均勻
  - T=1 相當於沒有 temperature
  - T=0 機率分布集中

token	logits
A	3.0
B	1.0
C	0.0
D	-1.0
E	-2.0

token	logits	Probability after softmax		
		T=0	T=1	T=2
A	3.0	0.99995	0.826	0.553
B	1.0	0.000045	0.112	0.203
C	0.0	~0	0.041	0.123
D	-1.0	~0	0.015	0.075
E	-2.0	~0	0.006	0.045

LLM 的 token 挑選秘辛

topK, topP, minP

介紹 (1/2)

- LLM 的模型預測分布
  - 前面為高機率合理 token，後面為低機率較偏差的 token
- 若直接使用模型預測出的機率分布→可能選到偏離 token
- 導致錯誤語法、話題不連貫等結果

今天天氣真→ LLM →

合理

偏離

token	原始機率
好	0.41
晴	0.25
壞	0.14
庫	0.07
盤	0.06
腦	0.04
狗	0.03

有20%的機率抽到品質差的 token

介紹 (2/2)

- 目標: 改變 token 的機率分布→生成品質控制

怎麼定義品質好?

- 雖然運算時看機率，但實際上過濾出的結果是 logits
  - 後續步驟會使用到 softmax，調整 logits 為機率總和為 1 的機率分布，避免抽選時有空洞機率

token	原始機率
好	0.41
晴	0.25
壞	0.14
庫	0.07
盤	0.06
腦	0.04
狗	0.03

機率總和=1

filtering →

Filter後	
0.41	
0.25	
0.14	
0.0	X
0.0	X
0.0	X
0.0	X

機率總和≠1

Next step:
Use softmax
to convert logits
→

最終機率
0.51
0.31
0.18
0.0
0.0
0.0
0.0

機率總和=1

Top-K

- 取前 K 個機率最高的 token

優點

- 簡單
- 每次取得數量固定

缺點

- 無法根據不同分布形狀調整
- 分布很尖時易取得偏離 token

token	原始機率
好	0.41
晴	0.25
壞	0.14
庫	0.07
盤	0.06
腦	0.04
狗	0.03

filtering →

K=3	K=6
0.41	0.41
0.25	0.25
0.14	0.14
0.0	0.07
0.0	0.06
0.0	0.04
0.0	0.0

Min-P

- 取機率 ≥ Pmax * P 的 token
- 相對最大機率去篩選掉低機率 token（相對品質的概念）

優點

- 可根據不同分布形狀調整

缺點

- 對平坦分布過濾的效果較弱

token	原始機率
好	0.41
晴	0.25
壞	0.14
庫	0.07
盤	0.06
腦	0.04
狗	0.03

filtering →

≥ 0.41*0.1=0.041

P=0.10
0.41
0.25
0.14
0.07
0.06
0.0
0.0

≥ 0.41*0.04=0.0164

P=0.04
0.41
0.25
0.14
0.07
0.06
0.04
0.03

Top-P

目前主流方法

- 保留 token 直至累積機率 ≥ P 的集合（含達到條件的那個 token）
  - 累積機率: 機率分布由大排到小後，從最大機率開始往下加的總和

優點

- 可根據不同分布形狀調整
- 分布很尖時不易取得偏離 token

缺點

- 較複雜
- 邊界不穩定 e.g. P=0.8 和 0.79 可能差很多 token
- 分布平坦時容易取得極大量 token

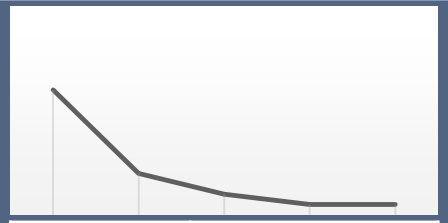
token	原始機率	累積機率
好	0.41	0.41 (0.41)
晴	0.25	0.66 (0.41+0.25)
壞	0.14	0.80 (0.41+0.25+0.14)
庫	0.07	0.87 (0.41+0.25+0.14+0.07)
盤	0.06	0.93 (0.41+0.25+...+0.06)
腦	0.04	0.97 (0.41+0.25+...+0.04)
狗	0.03	1.00 (0.41+0.25+...+0.03)

filtering →

P=0.8	P=0.96
0.41	0.41
0.25	0.25
0.14	0.14
0.0	0.07
0.0	0.06
0.0	0.04
0.0	0.0

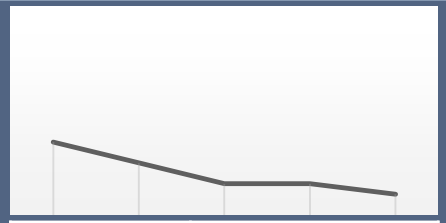
LAB-3

- 目標: 觀察並了解 sampling
- 實作 top-K, min-P, top-P
- 並根據右方 3 個分布得到過濾結果



token	原始機率
A	0.60
B	0.20
C	0.10
D	0.05
E	0.05

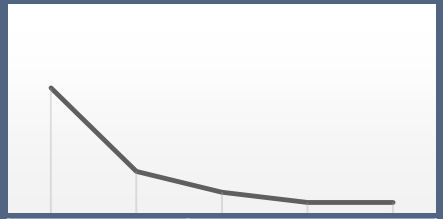
token	原始機率
A	0.22
B	0.21
C	0.20
D	0.19
E	0.18



token	原始機率
A	0.35
B	0.25
C	0.15
D	0.15
E	0.10

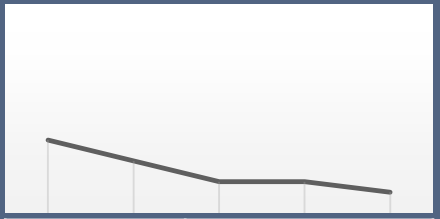
	Threshold	Result	Result	Result
Top K	1			
	3			
Min P	0.1			
	0.2			
Top P	0.5			
	0.85			

LAB-3 分析



token	原始機率
A	0.60
B	0.20
C	0.10
D	0.05
E	0.05

token	原始機率
A	0.22
B	0.21
C	0.20
D	0.19
E	0.18



token	原始機率
A	0.35
B	0.25
C	0.15
D	0.15
E	0.10

	Threshold	Result	Result	Result	analysis
Top K	1	A	A	A	看數量→分布差異對結果小
	3	ABC	ABC	ABC	
Min P	0.1	ABC	ABCDE	ABCDE	較為平坦的分布過濾效果差
	0.2	AB	ABCDE	ABCDE	
Top P	0.5	A	ABC	AB	可自適應截斷尾巴
	0.85	ABC	ABCDE	ABCD	

LLM 的 token 挑選秘辛

sample

介紹

- 目標: 拿到一個機率分布以後，挑選合適的 token 作為生成結果

怎麼挑選才合適?

今天天氣真→ LLM →

token	機率
好	0.25
晴	0.22
壞	0.18
陰	0.15
差	0.11
棒	0.06
讚	0.05

→ 晴

Random

目前主流方法

- 把拿到的機率分布當作抽樣機率，隨機進行挑選

優點

- 簡單直覺
- 保留全部可能性

缺點

- 隨機性無法控制，生成內容可能很有趣也可能很糟
- 低機率 token 可能被選，導致偏離上下文
- 生成預測性低，難保證一致性

token	機率
好	0.25
晴	0.22
壞	0.18
陰	0.15
差	0.11
棒	0.06
讚	0.05

抽樣機率

→ Random decoding → 晴

Greedy

- 但我們也可以透過調整參數的方式直接選擇機率最高的 token，不進行抽樣
- e.g. top K 將 K 設為 1，temperature 設為 0

優點

- 簡單直覺
- 穩定
- 不會有低機率 token 的問題

缺點

- 因只取最大，不考慮低機率 token，故缺乏多樣性
- 生成結果容易單調、重複

token	機率
好	0.25
晴	0.22
壞	0.18
陰	0.15
差	0.11
棒	0.06
讚	0.05

MAX

→ 好

LAB-4

- 目標: 了解並觀察不同策略的差異

- 透過調整參數實現 greedy
  - random 即為架構中預設的 decoding method
- 請根據右方的分布，用不同策略挑選 3 次 token
  - 2 個策略各 10 次

- 兩個不同策略的效果如何?

token	機率
A	0.25
B	0.22
C	0.18
D	0.15
E	0.11

LLM 的 token 挑選秘辛

penalty

功用

- 模型有時候會重複取用相同的 token
- 對於重複的 token 給予懲罰，降低下次挑到的可能性
- → 調整 logits

- 把右方這項定義為特定 token 的 logit: $logit(token)$

```
[OUTPUT]: 我是機器人， coded 的我，
在電腦裡，無情地運行。
我能夠計算，能夠記憶，
但我缺乏情感，缺乏心跳。

我能夠移動，能夠工作，
但我無法感受到，無法體驗。
我是機器人，-coded 的我，
在電腦裡，無情地運行。

我能夠學習，能夠改進，
但我無法理解，無法感受到。
我是機器人， coded 的我，
在電腦裡，無情地運行。
```

Repetition Penalty (1/2)

- 比例縮小出現過的 token 的 logit
- 實務上設定需 ≥ 1 → 1 表示沒有懲罰

為什麼要分正負做不同運算?

$$logit(token) = \begin{cases} \mathrm{I} * logit(token) \div repetition_penalty \ if \ logit(token) > 0 \\ \mathrm{I} * logit(token) * repetition_penalty \ if \ logit(token) \leq 0 \end{cases}$$

- I 是個 binary 數值，1 表示有出現過，0 表示沒出現過

LLM 生成過程:
早上陰陰的，跟昨天差不多，
但下午天氣好像會很

假設$repetition_penalty = 1.2$

token	logits	出現次數	懲罰後的 logits
好	2.0	1	2.0/1.2
晴	1.0	0	1.0
壞	0.5	0	0.5
陰	0.0	2	0.0*1.2
差	-0.5	1	-0.5*1.2
棒	-1.0	0	-1.0
讚	-2.0	0	-2.0

Repetition Penalty (2/2)

$$logit(token) = \mathrm{I} * logit(token) \div repetition_penalty \; if \; logit(token) > 0$$

- 假設不分運算都用除的方式縮小，且 repetition_penalty=2
  - 正數: 5/2=2.5 → 縮小
  - 負數: -5/2=-2.5 → 放大

反而增加 logit！

$$logit(token) = \begin{cases} \mathrm{I} * logit(token) \div repetition_penalty \; if \; logit(token) > 0 \\ \mathrm{I} * logit(token) * repetition_penalty \; if \; logit(token) \leq 0 \end{cases}$$

- 若依照正負區分運算方式
  - 正數: 5/2=2.5 → 縮小
  - 負數: -5*2=-10 → 縮小

Frequency Penalty

- 依照 token 的出現頻率進行懲罰
- 出現越頻繁懲罰越高

設為 0 表示沒有懲罰

$$logit(token) = logit(token) - frequency_penalty * count(token)$$

- $count(token)$ 表示特定 token 出現的數量

LLM 生成過程:
早上陰陰的，跟昨天差不多，
但下午天氣好像會很

假設 $frequency_penalty = 0.5$

token	logits	出現次數	懲罰後的 logits
好	2.0	1	2.0-0.5*1
晴	1.0	0	1.0-0.5*0
壞	0.5	0	0.5-0.5*0
陰	0.0	2	0.0-0.5*2
差	-0.5	1	-0.5-0.5*1
棒	-1.0	0	-1.0-0.5*0
讚	-2.0	0	-2.0-0.5*0

Presence Penalty

- 出現過的 token 會被懲罰
- 出現次數不同的 token 懲罰程度都相同

設為 0 表示沒有懲罰

$$logit(token) = logit(token) - presence_penalty * I$$

- I 是個 binary 數值，1 表示有出現過，0 表示沒出現過

LLM 生成過程:
早上陰陰的，跟昨天差不多，
但下午天氣好像會很

假設 $presence_penalty = 0.5$

token	logits	出現次數	懲罰後的 logits
好	2.0	1	2.0-0.5*1
晴	1.0	0	1.0-0.5*0
壞	0.5	0	0.5-0.5*0
陰	0.0	2	0.0-0.5*1
差	-0.5	1	-0.5-0.5*1
棒	-1.0	0	-1.0-0.5*0
讚	-2.0	0	-2.0-0.5*0

LAB-5

- 目標: 觀察不同 penalty 的效果

- 實作 repetition、frequency, presence penalty
- 分別應用 3 個 penalty 到下表的 logits，觀察變化並預測下個生成的 token
- 目前模型生成內容: ACA

token	logits	Repetition		Frequency		Presence	
		1.2	2.0	1.2	2.0	1.2	2.0
A	5						
B	2						
C	-0.3						
D	-1						
E	-1.5						

LAB-5 分析

- Penalty 數值調高會有較高的抑制效果
- Frequency 是 1 時，frequency penalty 跟 presence penalty 的效果相同
- Frequency penalty 會依照頻率增加而增強抑制
- Presence penalty 不管頻率高低的抑制效果都相同
- Repetition penalty 用比例縮小，抑制效果較平滑

- 目前模型生成內容: ACA

token	logits	Repetition		Frequency		Presence	
		1.2	2.0	1.2	2.0	1.2	2.0
A	5	4.17	2.5	2.6	1	3.8	3
B	2	2	2	2	2	2	2
C	-0.3	-0.36	-0.6	-1.5	-2.3	-1.5	-2.3
D	-1	-1	-1	-1	-1	-1	-1
E	-1.5	-1.5	-1.5	-1.5	-1.5	-1.5	-1.5

LLM 的挑戰

LLM 的挑戰

- 訓練資料偏見問題
  - 若資料中包含主觀意識，模型可能有偏見或刻板印象
- Hallucination
  - 模型答覆很肯定，但實際上他說明的東西是錯的或者不存在
  - 模型是生成最合適的答覆，而非最真實的
- 難以解釋
  - LLM 規模龐大又複雜，導致決策過程缺乏透明度
- 運算需求量大
  - LLM 模型常包含大量參數，需要高運算的 GPU 等資源
- 生成品質控制需求
  - 生成結果可能遇到多樣性低、偏離等問題

→ 本次課程介紹內容

LAB-Summary

- 請將前幾個 lab 的實作都串到迭代生成的架構中，使用以下題目推論
- 每個題目有不同的生成限制，每次僅能調整打勾的部分，請運用今天所學的所有策略推論
- 記錄設定&推論結果，每個策略都需嘗試至少兩個設定值，並說明你的發現

- 題目: 用一句話形容今天的心情
  - 無, Temperature 設定

generate	temperature	greedy/random	topK, topP, minP	penalty
迭代生成	V	random	X	X

- 題目: 請續寫一句話故事 “今天早上我打開冰箱，發現...”
  - 無, Top K, Top P, Min P

generate	temperature	greedy/random	topK, topP, minP	penalty
迭代生成	X	random	V	X

- 題目: 請列出三種奇怪的食物組合
  - Random, greedy

generate	temperature	greedy/random	topK, topP, minP	penalty
迭代生成	X	V	X	X

- 題目: 請寫 150 字的 rap 歌詞
  - Repetition, presence
  - Frequency, 無

generate	temperature	greedy/random	topK, topP, minP	penalty
迭代生成	X	random	X	V

LAB-Summary (advance)

- 目標: 了解並活用生成控制技巧

- 使用今天介紹的內容組成三個調整策略，以生成三個不同風格且你覺得最好的 100 字故事
- 紀錄策略&推論結果，並說明你的發現跟策略設計理由
- 要求:
  - 迭代生成
  - Temperature 可自由使用
  - 使用 random strategy
  - 自由使用 topK, topP, minP，種類不限
  - 自由使用 penalty，種類不限
- 題目: 如果貓會說話，他會怎麼用 rap 的方式跟你介紹他的世界?

Thank you